

CS426 QA3 — SSA recap and IR

CS 426 — Compiler Construction · Sep 30, 2026 at 4:11 PM · 10:00

Summary

SSA node convergence requires placing a phi function at the first node on a path from V to B' that satisfies the convergence condition. If convergence occurs at a node earlier than B', the phi function must be placed at that earlier node. To convert an AST to three-address code, constants and variables are loaded into virtual registers. Expressions are then processed recursively, such as computing E1 and E2 into temporary registers before performing the final operation.

Key concepts

- phi function — A special function used in SSA form to handle variable versions at join points.
- convergence condition — The condition that determines the location for placing a phi function on a path.
- three-address code — An intermediate representation where each instruction involves at most three operands.
- AST — An Abstract Syntax Tree representing the abstract syntactic structure of source code.
- virtual register — An abstraction of a CPU register allowing for an infinite number of storage locations.

Flagged in the lecture

- The quiz format is similar to the previous one.
- In quiz questions, the word 'can' implies finding at least one instance is sufficient.
- New symbols should be assumed to be not equivalent.

Takeaways

0:12–3:17 SSA node convergence

The first node on the path from V to B' that satisfies the convergence condition is B'. If convergence occurs earlier, the phi function must be placed at that earlier node.

- The first node on the path from V to B prime that satisfies the convergence condition is B prime.
- If convergence occurs at a node earlier than B prime, the phi function must be placed at that earlier node.
- The phi function should not be placed at node Z if convergence happens sooner on the path.
- Announcement: Corrected slides will be posted on the website.

3:17–6:46 Announcements: Quiz format and wording

The quiz is similar to the previous one. The word 'can' in questions implies that finding at least one instance is sufficient.

- Announcement: The quiz format is similar to the previous one.
- In quiz questions, the word 'can' implies that finding at least one instance is sufficient.
- New symbols should be assumed to be not equivalent.

6:46–9:59 AST to three-address code conversion

AST expressions are converted to three-address code by loading constants or variables into virtual registers and recursively computing operations like addition using temporary registers.

- AST expressions are converted to three-address code IR by following the structure of the expression.
- For a constant n , the value is loaded into a virtual register.
- For a variable x , its value is loaded into a virtual register.
- For an expression $E1 + E2$, $E1$ is computed and stored in register $T1$, $E2$ is computed and stored in register $T2$, and the sum is stored in a new register.
- Virtual registers are assumed to be infinite, so a new register is created for every new value.

Transcript

0:07 So I want to correct something wrong in the slides. I'll post the corrected slides. So here it should be B prime. B prime is the first node on the path from V to B prime that satisfies 0.2 node Z
0:28 should be E
0:32 what that means is like
0:34 there is no earlier node in which it converges right at
0:41 the path that goes from B to B
0:44 so if there exists another node at which it converges then you need to put the fee function at that node not on B prime so it should not be Z here it should be B prime
0:56 that was a small mistake on the side so so I'll put the corrected slides on the website
1:08 okay so now is a good time since we didn't do many quiz last time around so maybe we can do a start with one
1:23 should increase five I think yeah
3:01 but uh it actually starts when you click on it so
3:17 I think this is the same as the last quiz right right so so whenever I have new symbols like assume they are uh uh not equivalent
3:35 and also like in this uh
3:38 see the wording it can it says can right so there may be instances
3:47 this may be wrong but can means like you need to find one
4:11 increase all you might want to
4:36 the following is not true oh oh yeah so you did
4:47 the mini quiz so you might want to do it
5:21 then we could start to deflect
6:28 I think this is a good place anybody still doing it
6:34 okay
6:36 any questions on the quiz
6:40 any any doubts
6:44 so so here you can easily construct
6:49 a basic block structure where you have A goes to C, C goes to B, and then all of this will be true, right? And the second one is not true because if they are, if the original, if you had a diamond structure, I think I drew that in one of the classes, you don't necessarily need a phenode, right?
7:13 There may be multiple values entering the particular basic block from multiple predecessors or DL. Okay, but you don't need a phenode. So we had a mini quiz, so you might want to do it. But
7:31 yeah, so if there are any other questions,

7:33 I'll talk about it towards the end of the class. Okay, so another small correction I made in the slides. So this particular thing was Zast time around. So this should be B prime, right? That is a correction I made.

8:00 Corrected slides

8:02 hopefully today. Okay, any questions before I start today's lecture. Okay, so the goal, if you can remember, we stopped the last lecture trying to convert AST expressions into three-address code IR, right? So essentially, given a ST expression, can we convert that into three address code form with the instructions that I have shown you here?

8:41 So the strategy for doing that is like we need to look at the structure of the expression. If the expression is just a constant n , then we just load the value into a virtual register. That's pretty easy. If the expression is a variable x , then we load its values into a virtual. So in the first case, you load the constant value, and in the second case, you load the value of the variable.

9:08 And if the expression is of the form E_1 , some operation and E_2 , and in this case, we only allow plus. So, if it's of the form E_1 plus E_2 , you compute the left sub-expression E_1 , store the result in register T_1 , and compute the right sub expression E_2 and store its result in register T_2 , and then the final value would be just the addition of the

9:36 values in T_1 and T_2 , and you need to create another register $rest$ to do to store it. So, at this point in virtual register in virtual registers, you won't need to care about whether there exist enough registers because these registers are virtual. So, you can assume there are infinitely many registers. So, whenever you have a new value to store, you create a new one.